

Cloud Provider Analytics

Documento de diseño v1 · Primera evaluación parcial: diseño y fundación de datos

Arquitectura Lambda para ingerir, limpiar, conformar y publicar datos de uso, costo, facturación, soporte y GenAI de un proveedor cloud, con PySpark, Structured Streaming, Parquet y Cassandra/AstraDB.

Índice

0. Resumen ejecutivo	3
1. Interpretación del problema, usuarios, preguntas y objetivos medibles (consigna, Sección 5.2.1)	3
1.1 Problema	3
1.2 Usuarios y preguntas de negocio	3
1.3 Objetivos medibles (criterios de éxito)	3
2. Justificación de Big Data con las 5V (consigna, Sección 5.2.2)	4
3. Inventario y perfil inicial de fuentes (consigna, Sección 5.2.3)	4
4. Arquitectura de alto nivel v1 (consigna, Sección 5.2.4)	5
5. Patrón elegido y justificación (consigna, Sección 5.2.5)	6
6. Mapeo de requisitos a componentes y relación 5V ↔ decisiones (consigna, Sección 5.2.6)	6
7. Diseño del Data Lake (consigna, Sección 5.2.7)	7
7.1 Zonas, formato y reglas de promoción	7
7.2 Tablas, particiones y modo de escritura	7
7.3 Naming y convenciones	8
7.4 Reglas de calidad iniciales (derivadas del perfil)	8
7.5 Retención	8
7.6 Metadatos y linaje	8
8. Flujos de datos batch y streaming (consigna, Sección 5.2.8)	9
8.1 Flujo batch (maestros, facturación, tickets, NPS, marketing)	9
8.2 Flujo streaming (<code>usage_events_stream</code>)	9
8.3 Late data: hallazgo crítico	9
9. Flujo batch de referencia con lógica MapReduce (consigna, Sección 5.2.9)	10
9.1 Diseño en dos jobs encadenados	10
9.2 Evidencia de ejecución (<code>evidence/mapreduce/mapreduce_stats.json</code>)	10
10. Supuestos, riesgos, mitigaciones y decisiones abiertas (consigna, Sección 5.2.10)	11
10.1 Supuestos	11
10.2 Riesgos y mitigaciones	11
10.3 Decisiones abiertas (a cerrar con el feedback de la Entrega 1)	11
10.4 Modelo de serving preliminar (<i>query-first</i> , a validar en la Entrega 2)	12
11. Estimación preliminar de esfuerzo, roles y recursos (consigna, Sección 5.2.11)	12
11.1 Roles	12
11.2 Esfuerzo estimado (horas)	12
11.3 Recursos	12
12. Repositorio y evidencia (consigna, Sección 5.2.12)	12
Próximos pasos (hacia la Entrega 2)	13

Este documento cubre los 12 puntos del alcance obligatorio (Sección 5.2 de la consigna). Cada sección indica a qué punto responde. Todas las cifras sobre los datos salen de `evidence/profile/profile_report.md`, que genera `src/profiling/profile_landing.py` y se puede volver a generar.

0. Resumen ejecutivo

Tema	Decisión v1
Problema	Publicar datos confiables de uso, costo, facturación, soporte y GenAI de clientes de un proveedor cloud para FinOps, Soporte y Producto.
Patrón	Lambda con lógica compartida: <i>streaming</i> (Spark Structured Streaming) para <code>usage_events_stream</code> ; <i>batch</i> diario/mensual para maestros, facturación, tickets y encuestas.
Data Lake	Landing (inmutable) → Bronze → Silver → Gold en Parquet , más <code>quarantine/</code> , <code>_checkpoints/</code> y <code>_metadata/</code> .
Procesamiento	PySpark 4.x (DataFrame API) en Google Colab / local <code>local[*]</code> .
Serving	Cassandra/AstraDB con tablas <i>query-first</i> , una por consulta obligatoria.
Hallazgo crítico	Los eventos llegan desordenados : cada archivo trae eventos de los 60 días. Con un watermark de 1 día se descartaría el 97,5 % de los eventos. Ver Sección 8.3 y la decisión D-05.
Evidencia	Perfil reproducible de las 8 fuentes y un job MapReduce de referencia cuya salida coincide al 100 % con su versión Spark (11.050 claves).

1. Interpretación del problema, usuarios, preguntas y objetivos medibles (consigna, Sección 5.2.1)

1.1 Problema

El área de datos del proveedor cloud recibe datos crudos con nulos, tipos ambiguos, costos negativos, outliers y un cambio de esquema a mitad del histórico (v1 → v2 el 18/07/2025, que agrega `carbon_kg` y `genai_tokens`). Hoy no existe una vista única y confiable que combine **uso casi en tiempo real** con **maestros y facturación**. El proyecto debe ingerir, limpiar, conformar y publicar esos datos para el consumo analítico.

1.2 Usuarios y preguntas de negocio

Usuario	Dominio	Preguntas principales	Latencia que necesita
Analista FinOps	FinOps	¿Cuánto cuesta y cuánto consume cada organización por servicio y por día? ¿Cuáles son los top-N servicios por costo en los últimos 14 días? ¿Qué costos son anómalos? ¿Cuál es el revenue mensual neto en USD?	Minutos para el costo incremental; D+1 para revenue
Líder de Soporte	Soporte	¿Cómo evolucionan los tickets críticos y la tasa de SLA breach por día (últimos 30 días)? ¿Cuál es el CSAT por organización?	D+1
Product Manager	Producto / GenAI	¿Cuánto se usa cada servicio (requests, CPU, storage)? ¿Cuántos tokens GenAI se consumen por día y a qué costo? ¿Cuánto carbono se emite?	Horas / D+1
Ingeniería de datos (autora)	Plataforma	¿El pipeline corrió, cuántos registros entraron y cuántos quedaron en quarantine? ¿Es reprocesable?	Por corrida

1.3 Objetivos medibles (criterios de éxito)

ID	Objetivo	Métrica / umbral	Se verifica en
O1	Frescura del uso	Evento → Gold <code>org_daily_usage_by_service</code> en ≤ 5 min (trigger de 1 min)	Entrega 2 (logs de streaming)
O2	Frescura batch	Maestros y facturación disponibles en Gold antes de las 06:00 (D+1)	Entrega 2
O3	Unicidad	0 <code>event_id</code> duplicados en Silver/Gold, incluso tras re-ejecutar	Conteos antes/después
O4	Calidad trazable	100 % de los registros terminan en Silver o en <code>quarantine</code> con motivo (no hay pérdida silenciosa)	<code>_metadata/run_log</code>
O5	Trazabilidad	100 % de las filas de Bronze tienen <code>ingest_ts</code> , <code>source_file</code> y <code>run_id</code>	Reglas de calidad
O6	Serving	Las 5 consultas obligatorias se resuelven con una sola partición de Cassandra cada una	CQL + capturas
O7	Reproducibilidad	Pipeline ejecutable desde un entorno limpio con el Quickstart en ≤ 15 min	README

2. Justificación de Big Data con las 5V (consigna, Sección 5.2.2)

La muestra entregada (13 MB, 43.200 eventos) entra en memoria y pandas la procesa sin problemas. La justificación no se apoya en la muestra sino en el sistema real que la muestra representa. Por eso se usa el mismo código Spark en modo local, y escala sin reescribirse.

V	Evidencia en la muestra	Proyección a un proveedor real	Implicancia de arquitectura
Volumen	43.200 eventos / 60 días = 720 eventos/día sobre 400 recursos (~295 bytes por evento JSON).	Supuesto: 50.000 orgs × 40 recursos × 1 evento/min ≈ 2.900 M eventos/día ≈ 850 GB/día en JSON (≈ 85-170 GB/día en Parquet comprimido) y ~300 TB/año.	Almacenamiento distribuido y columnar (Parquet), particionado por fecha, procesamiento <i>scale-out</i> con Spark.
Velocidad	Los eventos llegan en 120 micro-archivos; el costo es incremental (<code>cost_usd_increment</code>).	Flujo continuo; FinOps necesita detectar picos de costo en minutos, no al cierre de mes.	Structured Streaming con micro-batches, checkpoints y escrituras idempotentes.
Variedad	CSV tabulares, JSONL semi-estructurado con dos versiones de esquema , JSON embebido en CSV (<code>tags_json</code>) y texto libre (<code>comment</code> de NPS).	Más fuentes (CRM, billing, logs).	Esquemas explícitos, capa Silver de conformación y compatibilidad v1/v2.
Veracidad	1.309 <code>value</code> como string, 877 <code>value</code> nulos, 2.075 <code>unit</code> nulos, 216 costos negativos, 312 outliers de costo, CSAT fuera de rango (40), <code>last_login < created_at</code> (232), FX de USD ≠ 1.	Igual o peor a escala.	Reglas de calidad verificables, quarantine en Parquet, métricas por corrida.
Valor	Mart diario de uso/costo, revenue en USD, SLA y GenAI.	Ahorro por detección temprana de anomalías de costo, prevención de churn (SLA/NPS) y adopción de GenAI.	Gold orientado a dominios + serving <i>query-first</i> en Cassandra.

También es relevante la **variabilidad**: la distribución cambia en el tiempo (aparecen `carbon_kg` y `genai_tokens` en v2) y eso afecta tanto las reglas de calidad como los modelos de anomalías.

3. Inventario y perfil inicial de fuentes (consigna, Sección 5.2.3)

Detalle completo por columna en `evidence/profile/profile_report.md` y diccionario en `02_inventario_fuentes.md`.

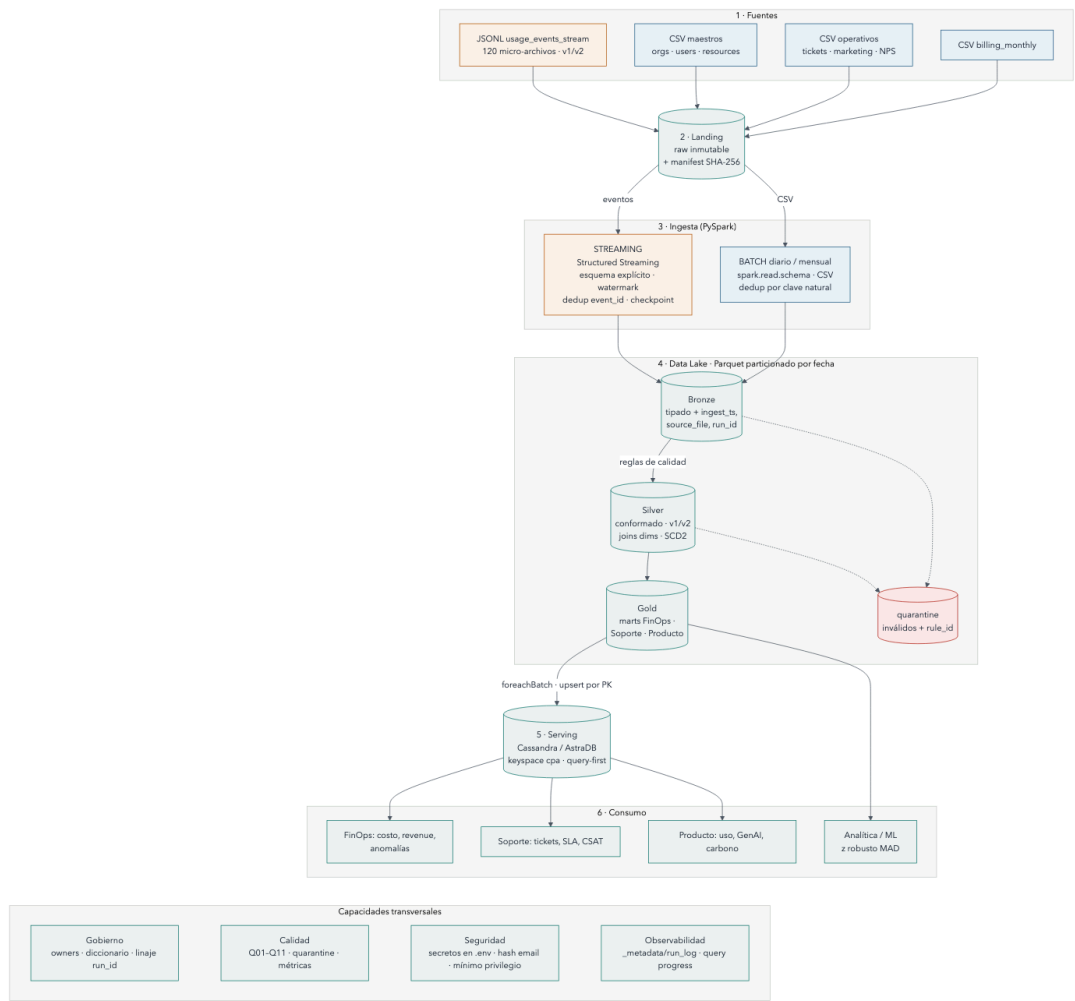
Fuente	Grano (1 fila ⇒)	Frecuencia / modo	Filas	Clave natural	Calidad: hallazgos principales	Trazabilidad	Riesgo
<code>customers_orgs.csv</code>	organización	snapshot batch (diario)	80	<code>org_id</code>	11 <code>nps_score</code> nulos; 1 fuera de [-100,100]; 25 orgs con <code>plan_tier</code> incoherente con <code>is_enterprise</code>	sin timestamp de actualización → requiere SCD2 por hash	Medio
<code>users.csv</code>	usuario	snapshot batch	800	<code>user_id</code>	139 <code>last_login</code> nulos; 232 con <code>last_login < created_at</code> ; <code>email</code> es PII	<code>created_at</code>	Medio (PII)
<code>resources.csv</code>	recurso cloud	snapshot batch	400	<code>resource_id</code>	83 <code>tags_json</code> nulos; JSON embebido; 85 recursos con <code>pii:true</code>	<code>created_at</code>	Bajo
<code>support_tickets.csv</code>	ticket	batch diario (cambia estado)	1.000	<code>ticket_id</code>	240 abiertos (sin <code>resolved_at</code>); 254 CSAT nulos; 40 CSAT fuera de [1,5]	<code>created_at</code> / <code>resolved_at</code>	Medio
<code>marketing_touches.csv</code>	interacción	batch diario	1.500	<code>touch_id</code>	96 <code>converted=True</code> sin <code>clicked=True</code>	<code>timestamp</code> (fecha)	Bajo
<code>nps_surveys.csv</code>	encuesta org-fecha	batch	92	<code>org_id+survey_date</code>	19 <code>nps_score</code> nulos; 10 comentarios nulos	<code>survey_date</code>	Bajo

Fuente	Grano (1 fila =)	Frecuencia / modo	Filas	Clave natural	Calidad: hallazgos principales	Trazabilidad	Riesgo
billing_monthly.csv	factura org-mes	batch mensual	240	invoice_id	137 credits nulos; 13 subtotales negativos ; 3 monedas; FX de USD entre 0,85 y 1,12 ; 50 orgs facturan en más de una moneda	month	Alto (semántica FX)
usage_events_stream/*.jsonl	evento de uso	streaming (120 micro-archivos x 360)	43.200	event_id	1.309 value string; 877 value nulos; 2.075 unit nulos; 216 costos < 0 (211 < -0,01); 312 outliers; esquema v1/v2; llegada desordenada	timestamp UTC, event_id, schema_version	Alto

Integridad referencial verificada: el 100 % de los `org_id` y `resource_id` de los eventos existen en los maestros; `service` y `region` de cada evento coinciden con los del recurso. No hay `event_id` duplicados en este corte, pero el diseño igual deduplica, porque reprocesar o re-entregar archivos sí genera duplicados.

Cobertura temporal: eventos 03/07–31/08/2025 (60 días × 720); facturación 06–08/2025 (junio no tiene eventos, así que la reconciliación entre uso y facturación sólo es posible para julio y agosto); tickets 09/05–31/08/2025.

4. Arquitectura de alto nivel v1 (consigna, Sección 5.2.4)



Fuente editable: `diagramas/arquitectura_v1.mmd` (Mermaid).

Responsabilidades por componente

Componente	Responsabilidad	Tecnología
Landing	Guardar los archivos tal como llegan, con hash SHA-256 para verificar que no cambian	Sistema de archivos / Google Drive (en producción: S3/GCS/HDFS)
Ingesta batch	Leer CSV con esquema explícito, agregar columnas técnicas, deduplicar por clave y escribir Bronze	PySpark <code>spark.read.schema(...)</code>
Ingesta streaming	Leer el directorio JSONL de forma incremental, aplicar watermark, deduplicar por <code>event_id</code> y hacer checkpoint	Spark Structured Streaming
Silver	Castear, normalizar, unificar v1/v2, hacer joins con dimensiones, tratar nulos y outliers, aplicar SCD2	PySpark
Gold	Construir los marts por dominio con granos explícitos	PySpark / Spark SQL
Serving	Publicar los marts en tablas <i>query-first</i> con upserts idempotentes	Spark Cassandra Connector o driver Python en <code>foreachBatch</code>
Transversales	Calidad, metadatos, linaje, seguridad, observabilidad	<code>_metadata/run_log</code> , reglas en config, <code>.env</code>

5. Patrón elegido y justificación (consigna, Sección 5.2.5)

Decisión: Lambda con una única base de código de transformaciones (D-01).

Criterio	Batch puro	Lambda (elegido)	Kappa
Latencia de costos de uso ($O1 \leq 5$ min)	✗ (D+1)	✓ streaming	✓
Maestros, facturación y NPS son snapshots pequeños y lentos	✓	✓ batch	✗: obliga a convertir CSV estáticos en streams sin beneficio
Facturación mensual con reconciliación	✓	✓	Complejo (re-stream completo)
Complejidad operativa	Baja	Media	Media-alta
Requisito invariable (streaming + batch)	✗	✓	✓ (forzado)

Por qué Lambda: la naturaleza de las fuentes ya está dividida. Los eventos son un flujo continuo y su valor cae con la demora (anomalías de costo). Los maestros y la facturación son lotes pequeños con cadencia diaria o mensual. Kappa nos haría tratar 80 organizaciones y 240 facturas como streams sin ganar nada.

Mitigación de la desventaja clásica de Lambda (dos códigos): en Spark, batch y streaming comparten la API de DataFrames. Las funciones de Silver/Gold (`conform_events(df)` , `build_daily_usage(df)`) se escriben una vez y se aplican tanto dentro de `foreachBatch` como en el *backfill* batch. La capa de *serving* es una sola (Cassandra) y el upsert por clave primaria reconcilia ambas rutas.

6. Mapeo de requisitos a componentes y relación 5V ↔ decisiones (consigna, Sección 5.2.6)

Leyenda: ● responsable principal · ○ participa. Componentes: **LND** Landing · **BIN** ingesta batch · **SIN** ingesta streaming · **BRZ** Bronze · **SLV** Silver · **GLD** Gold · **QTN** quarantine · **CAS** Cassandra · **META** metadatos/run_log · **CFG** config/secretos.

ID	Requisito (origen)	LND	BIN	SIN	BRZ	SLV	GLD	QTN	CAS	META	CFG	Evidencia prevista
R01	Datos crudos inmutables (consigna, Sección 3.2)	●								○		<code>landing_manifest.sha256</code> sin cambios
R02	Ingesta batch CSV → Bronze Parquet particionado, esquema explícito, columnas técnicas (consigna, Sección 4.4)	○	●		●					○	○	rutas + <code>printSchema</code>
R03	Streaming JSONL con esquema, watermark, dedup <code>event_id</code> , late data, checkpoint (consigna, Sección 4.4)	○		●	●			○		○	○	<code>_checkpoints/</code> , query progress
R04	Evolución de esquema v1/v2 (consigna, Sección 3.2)			○	○	●						conteos por <code>schema_version</code>
R05	Tipos ambiguos con cast y fallback controlado (consigna, Sección 3.2)		○	○	○	●		○				<code>value_raw</code> vs <code>value</code>
R06	Reglas de calidad + quarantine (consigna, Sección 4.4)				○	●		●		○	○	muestras de quarantine
R07	Normalización y joins con dimensiones (consigna, Sección 4.4, Silver)					●						tablas Silver
R08	Features: <code>daily_cost_usd</code> , <code>requests</code> , <code>cpu_hours</code> , <code>storage_gb_hours</code> , <code>genai_tokens</code> , <code>carbon_kg</code>					○	●					mart diario

ID	Requisito (origen)	LND	BIN	SIN	BRZ	SLV	GLD	QTN	CAS	META	CFG	Evidencia prevista
R09	Anomalías de costo (z robusto / MAD / percentiles)					○	●				○	cost_anomaly_mart
R10	Marts FinOps, Soporte, Producto (consigna, Sección 7.3)						●		○			5 marts
R11	Serving query-first en Cassandra (consigna, Sección 4.4)						○		●		○	CQL + capturas
R12	Idempotencia (consigna, Sección 4.4)		○	●	●	●	●		●	○		conteos antes/después
R13	Performance: particiones, coalesce, tamaños (consigna, Sección 4.4)		○	○	●	●	●			○		tamaños por partición
R14	Gobierno: metadatos, linaje, owners, seguridad (consigna, Sección 4.4)	○			○	○	○		○	●	●	run_log, DECISIONS.md
R15	Objetivos O1-O7 (Sección 1.3)		○	●			○		●	●		ver Sección 1.3

Relación entre las 5V y las decisiones

V	Decisión de arquitectura que motiva	Componentes
Volumen	Parquet columnar + particionado por fecha; Spark <i>scale-out</i> ; Cassandra particionada por org/fecha	BRZ, SLV, GLD, CAS
Velocidad	Structured Streaming con micro-batches y checkpoints; upsert idempotente en serving	SIN, CAS
Variedad	Esquemas explícitos por fuente; Silver unifica v1/v2; parseo de <code>tags_json</code>	BIN, SIN, SLV
Veracidad	Reglas de calidad declarativas, quarantine con motivo, métricas por corrida, dedup	SLV, QTN, META
Valor	Marts por dominio con granos explícitos y tablas por consulta	GLD, CAS

7. Diseño del Data Lake (consigna, Sección 5.2.7)

7.1 Zonas, formato y reglas de promoción

```

datalake/
├─ landing/                # raw inmutable (tal cual llega; sólo lectura para el pipeline)
├─ bronze/<entidad>/       # Parquet, mismo grano que la fuente + columnas técnicas
├─ silver/<dim_|fact_><entidad>/ # Parquet conformado
├─ gold/<mart>/            # Parquet orientado a consumo
├─ quarantine/<zona>_<entidad>/ # Parquet de registros rechazados + rule_id + motivo
├─ _checkpoints/<query>/   # estado de Structured Streaming
└─ _metadata/run_log/      # una fila por corrida y etapa: conteos, duración, estado

```

Promoción	Condición para promover	Lo que no cumple
Landing → Bronze	El archivo parsea con el esquema explícito; se agregan <code>ingest_ts</code> , <code>source_file</code> , <code>run_id</code> ; se deduplica por clave natural (eventos: <code>event_id</code>)	Líneas corruptas → <code>quarantine/bronze_<entidad></code> (modo <code>PERMISSIVE</code> + <code>_corrupt_record</code>)
Bronze → Silver	Pasa las reglas de calidad <i>bloqueantes</i> (tabla 7.4); tipos casteados; v1/v2 unificados; joins con dimensiones válidos	→ <code>quarantine/silver_<entidad></code> con <code>rule_id</code>
Silver → Gold	Partición de fecha cerrada (watermark superado o corrida batch completa) y reconciliación de conteos Silver ↔ Gold	La corrida queda en <code>FAILED</code> en el <code>run_log</code> ; Gold no se pisa
Gold → Cassandra	Gold escrito con éxito (<code>_SUCCESS</code>)	Reintento; el upsert por PK es idempotente

7.2 Tablas, particiones y modo de escritura

Zona	Tabla	Grano	Partición	Modo de escritura	Justificación
Bronze	<code>usage_events</code>	evento	<code>event_date</code>	append (stream)	Poda por fecha; el reproceso es por día; consistente con el grano de Gold
Bronze	<code>customers_orgs</code> , <code>users</code> , <code>resources</code>	fila del snapshot	<code>ingest_date</code>	overwrite dinámico de partición	Cada snapshot es reproducible; < 100 KB, no conviene particionar por otra cosa
Bronze	<code>support_tickets</code> , <code>marketing_touches</code> , <code>nps_surveys</code>	fila	<code>ingest_date</code>	overwrite dinámico	Idem
Bronze	<code>billing_monthly</code>	factura	<code>month</code>	overwrite dinámico	Cadencia mensual natural
Silver	<code>fact_usage_events</code>	evento limpio	<code>event_date</code>	overwrite dinámico por fecha afectada	Idempotente ante late data
Silver	<code>dim_org</code> (SCD2), <code>dim_user</code> , <code>dim_resource</code>	entidad-versión	sin partición	overwrite	Tamaño chico; se usa en <i>broadcast join</i>

Zona	Tabla	Grano	Partición	Modo de escritura	Justificación
Silver	fact_tickets, fact_billing, fact_nps, fact_marketing	fila	mes	overwrite dinámico	Volumen bajo
Gold	org_daily_usage_by_service	org × día × servicio	usage_date	overwrite dinámico	Grano obligatorio (Entrega 2)
Gold	revenue_by_org_month	org × mes	month	overwrite dinámico	Consigna, Sección 7.3
Gold	cost_anomaly_mart	org × día × servicio	date	overwrite dinámico	Consigna, Sección 7.3
Gold	tickets_by_org_date	org × día × severidad	date	overwrite dinámico	Consigna, Sección 7.3
Gold	genai_tokens_by_org_date	org × día	date	overwrite dinámico	Consigna, Sección 7.3

Control de archivos chicos: en la muestra, una partición diaria de eventos tiene ~720 filas, unos 25 KB en Parquet. Se aplica `coalesce(1)` por partición en Bronze/Silver/Gold para no generar cientos de *small files*. A escala real (~50 M eventos/día por partición) se pasaría a `repartition(n)` buscando archivos de 128–512 MB. No se particiona por `service` ni por `org_id`, porque multiplicaría por 6 y por 80 la cantidad de archivos.

7.3 Naming y convenciones

- Todo en `snake_case`, en inglés para tablas y columnas (como la fuente) y en español para documentación.
- Silver usa los prefijos `dim_` y `fact_`. Gold usa el nombre del mart de la Sección 7.3 de la consigna.
- Las columnas técnicas llevan prefijo fijo: `ingest_ts`, `source_file`, `run_id`, `record_hash`; en SCD2 se agregan `valid_from`, `valid_to`, `is_current`.
- Fechas en UTC (`spark.sql.session.timeZone=UTC`, ver D-07); `event_ts` es el timestamp y `event_date` la fecha derivada.
- Montos con sufijo de moneda: `*_usd`, `*_local`.

7.4 Reglas de calidad iniciales (derivadas del perfil)

ID	Regla	Tipo	Hallazgo que la motiva
Q01	<code>event_id</code> no nulo y único	Bloqueante → quarantine	Clave de idempotencia
Q02	<code>cost_usd_increment</code> ≥ -0.01 ; si es menor → quarantine con flag de anomalía	Bloqueante	211 eventos < -0.01 (mín. -154,46)
Q03	<code>unit</code> no nulo cuando <code>value</code> existe; si falta, se deriva de <code>metric</code>	Corrección	2.075 <code>unit</code> nulos; la relación <code>metric</code> ↔ <code>unit</code> es 1:1
Q04	<code>value</code> casteable a double; si no, <code>value=null</code> , se conserva <code>value_raw</code> y se marca el flag	Corrección	1.309 strings y 877 nulos
Q05	<code>org_id</code> y <code>resource_id</code> existen en las dimensiones	Bloqueante	Hoy 0 huérfanos; protege a futuro
Q06	<code>csat</code> dentro de [1,5]; si no, <code>null</code> + flag	Corrección	40 fuera de rango
Q07	<code>last_login</code> $\geq$ <code>created_at</code> ; si no, flag <code>login_inconsistente</code>	Advertencia	232 casos
Q08	<code>exchange_rate_to_usd</code> dentro de un rango plausible por moneda	Advertencia	USD con FX $\neq 1$ (ver D-06)
Q09	<code>subtotal</code> < 0 se marca como nota de crédito	Advertencia	13 casos
Q10	<code>credits</code> nulo → 0	Corrección	137 nulos
Q11	Outlier de costo: $ z \text{ robusto (MAD) por servicio+métrica} > 3,5$ → flag (no se descarta)	Feature / anomalía	312 eventos

7.5 Retención

Zona	Retención propuesta	Motivo
Landing	Indefinida (en producción: <i>cold storage</i> después de 90 días)	Es la fuente de verdad para reprocesar; inmutable
Bronze	13 meses	Reprocesos y comparativas interanuales
Silver	25 meses	Análítica histórica
Gold	36 meses	Reporting financiero
Quarantine	90 días	Diagnóstico y reinyección después de corregir
<code>_checkpoints</code>	Mientras exista la query; se borra solo en un <code>reset</code> controlado	Garantía <i>exactly-once</i> del sink

7.6 Metadatos y linaje

- **A nivel fila:** `ingest_ts`, `source_file` (`input_file_name()`), `run_id`, `schema_version` y `record_hash` (para SCD2 y para detectar cambios).
- **A nivel corrida:** `_metadata/run_log` (Parquet) con `run_id`, etapa, tabla, `rows_in`, `rows_out`, `rows_quarantine`, duración, estado y versión de código (git SHA).

- **Linaje:** cada tabla Gold documenta sus tablas Silver de origen en el diccionario. El `run_id` permite recorrer el camino Gold → Silver → Bronze → `source_file`.
- **Owners:** cada dominio tiene un responsable (ver Sección 11).

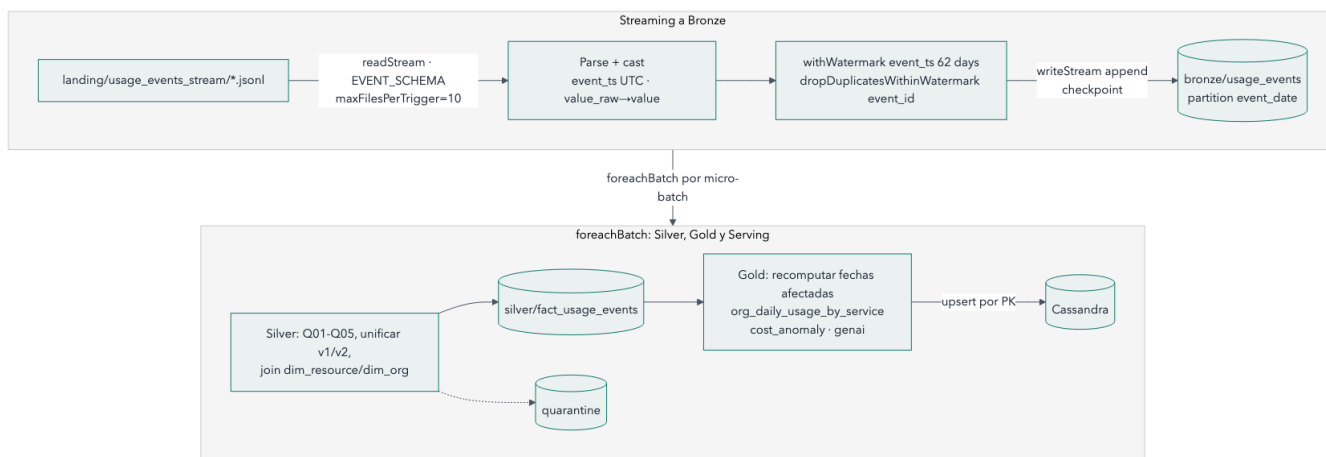
8. Flujos de datos batch y streaming (consigna, Sección 5.2.8)

8.1 Flujo batch (maestros, facturación, tickets, NPS, marketing)

Paso	Qué hace	Herramienta
1	Detectar archivos nuevos en <code>landing/</code> y registrar su hash	Python <code>hashlib</code> , <code>manifest</code>
2	Leer CSV con <code>StructType</code> explícito (sin <code>inferSchema</code>), <code>mode=PERMISSIVE</code>	<code>spark.read.schema().csv()</code>
3	Agregar <code>ingest_ts</code> , <code>source_file</code> , <code>run_id</code> ; deduplicar por clave natural	<code>withColumn</code> , <code>input_file_name</code> , <code>dropDuplicates</code>
4	Escribir Bronze Parquet particionado	<code>write.partitionBy().mode("overwrite")</code> con <code>partitionOverwriteMode=dynamic</code>
5	Silver: castear, normalizar, aplicar SCD2 en <code>dim_org</code> , convertir facturación a USD, aplicar reglas Q06-Q10	DataFrame API, <code>F.broadcast</code> para dimensiones
6	Gold: <code>revenue_by_org_month</code> , <code>tickets_by_org_date</code>	<code>groupBy/agg</code> , Spark SQL
7	Publicar en Cassandra (upsert) y escribir el <code>run_log</code>	Spark Cassandra Connector / <code>cassandra-driver</code>

Orquestación v1: un script `run_batch.py` con etapas idempotentes, ejecutable desde Colab. En producción se usaría Airflow (fuera de alcance).

8.2 Flujo streaming (`usage_events_stream`)



Fuente editable: `diagramas/flujo_streaming.mmd` (Mermaid).

- **Trigger:** `processingTime="1 minute"`; en la demo, `availableNow=True` para procesar los 120 archivos de forma determinística.
- **Simulación de micro-lotes:** `maxFilesPerTrigger=10` produce 12 micro-batches de ~3.600 eventos.
- **Checkpoint:** `_checkpoints/bronze_usage_events` y `_checkpoints/silver_usage`. Si el proceso se reinicia, no se reprocesan archivos ya confirmados.
- **Idempotencia del sink:** Bronze en modo append con checkpoint (*exactly-once* del file sink); Silver/Gold con `overwrite` dinámico de las particiones de fecha afectadas en cada micro-batch; Cassandra con upsert por clave primaria.

8.3 Late data: hallazgo crítico

El perfil muestra que **los 120 archivos contienen eventos de todo el rango 03/07–31/08**. La llegada no sigue el tiempo del evento. Simulamos un stream de un archivo por trigger y medimos qué fracción de eventos quedaría por detrás del watermark:

Delay del watermark	Eventos que serían "tarde"	%
1 hora	42.801	99,1 %
1 día	42.118	97,5 %
7 días	37.831	87,6 %
30 días	21.421	49,6 %
60 días	0	0 %

Consecuencia: un watermark "típico" (minutos u horas) haría que los operadores con estado descarten casi todo. **Propuesta (D-05, abierta):**

1. El stream a Bronze aplica `withWatermark("event_ts", "62 days") + dropDuplicatesWithinWatermark(["event_id"])`. El delay cubre el desorden observado (60 días) más 2 días de margen, así que con estos datos se pierde **0 %**. El estado queda acotado a ~43 k claves (unos pocos MB), viable para este volumen.
2. Si llega un evento más tarde que el watermark, Spark lo descarta en el operador con estado. Para que no se pierda en silencio, un batch diario de reconciliación compara las líneas por `source_file` en Landing (manifest) contra Bronze. Los faltantes se releen desde Landing y van a `quarantine/late_events` con su motivo.
3. Silver se procesa en `foreachBatch` y sobrescribe sólo las particiones de fecha que tocó cada micro-batch.
4. Gold se recalcula para las **fechas afectadas** en cada micro-batch (overwrite dinámico), así un evento tardío corrige el agregado del día que corresponde.

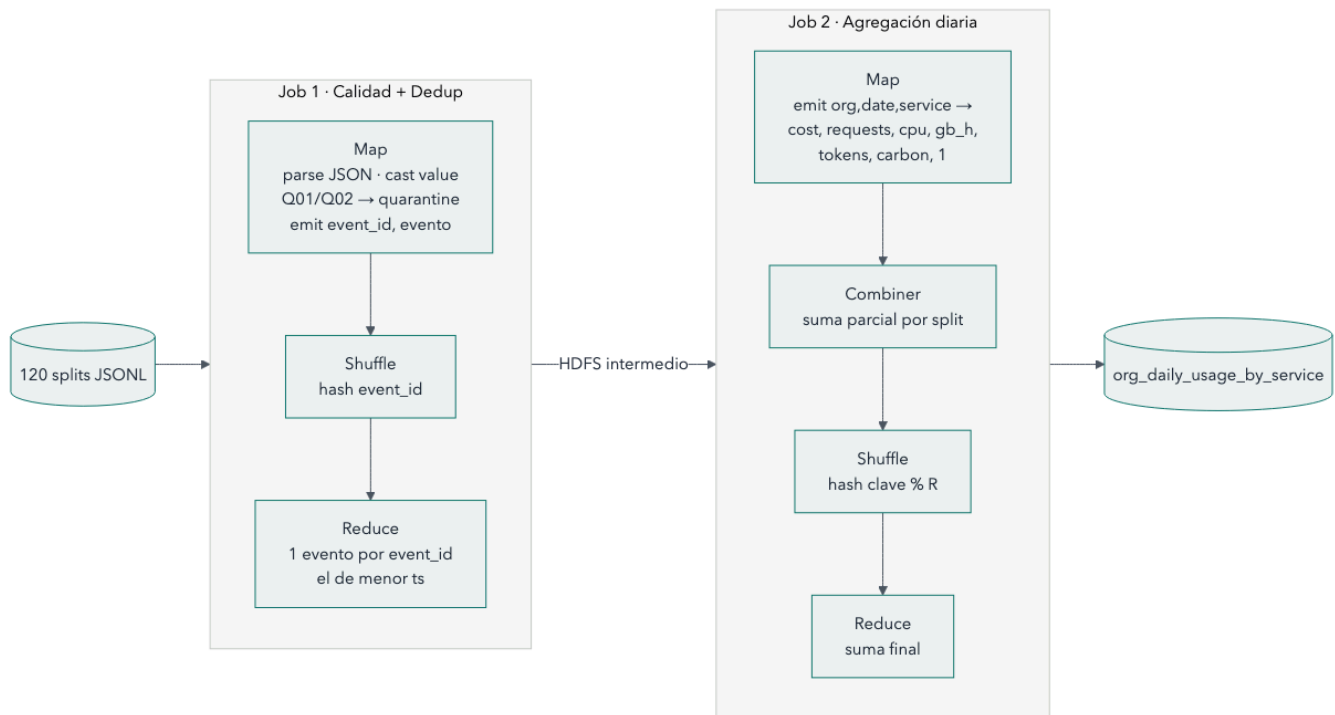
A escala real, un watermark de 62 días no sería sostenible. Allí la deduplicación se haría con un `merge` por `event_id` sobre Silver (Delta/Iceberg). Lo dejamos como decisión abierta para validar en la Entrega 4.

9. Flujo batch de referencia con lógica MapReduce (consigna, Sección 5.2.9)

Objetivo: calcular el `org_daily_usage_by_service` (`org × día × servicio`) a partir de los eventos crudos. Código:

`src/mapreduce/daily_usage_mapreduce.py` (simulación en Python puro de Hadoop MR) y `src/mapreduce/daily_usage_spark.py` (equivalente PySpark).

9.1 Diseño en dos jobs encadenados



Fuente editable: `diagramas/mapreduce_referencia.mmd` (Mermaid).

```

Job 1  map(line):      e = parse(line)
                        if invalid(e): emit("__quarantine__", motivo)
                        else: emit(e.event_id, clean(e))           # value "12.5" → 12.5
                        reduce(event_id, es): emit(event_id, min(es, key=ts)) # duplicados → quarantine

Job 2  map(e):         emit((e.org_id, date(e.ts), e.service),
                        {cost: e.cost, e.metric: e.value, genai_tokens, carbon_kg, events: 1})
                        combine(k, vs): emit(k, sum(vs))          # pre-agregación local: menos datos en el shuffle
                        reduce(k, vs): emit(k, sum(vs))
  
```

9.2 Evidencia de ejecución (`evidence/mapreduce/mapreduce_stats.json`)

Contador	Job 1	Job 2
Registros de entrada al map	43.200	42.989
Registros de salida del map	43.200	42.989

Contador	Job 1	Job 2
Registros en el shuffle	43.200	28.377 (el combiner los reduce un 34 %)
Registros de salida del reduce	43.200	11.050 claves org×día×servicio
Quarantine	211 <code>costo_invalido</code>	–

Validación cruzada: la salida MapReduce y la de PySpark coinciden en las **11.050 claves** y en todas las métricas (diferencia máxima < 1e-4, por redondeo). El plan físico de Spark (`evidence/mapreduce/spark_physical_plan.txt`) muestra la misma estructura: `HashAggregate(partial_sum)` (el combiner) → `Exchange hashpartitioning` (el shuffle) → `HashAggregate(sum)` (el reduce). Además hay un `Exchange` previo para `dropDuplicates(event_id)` (el Job 1).

Lección: en Hadoop, los dos jobs escriben su resultado intermedio a disco (HDFS replicado ×3). Spark ejecuta todo como un único DAG en memoria, que es la razón por la que elegimos Spark.

10. Supuestos, riesgos, mitigaciones y decisiones abiertas (consigna, Sección 5.2.10)

10.1 Supuestos

ID	Supuesto
S1	Los timestamps de eventos están en UTC (vienen con sufijo <code>Z</code>).
S2	<code>cost_usd_increment</code> ya está en USD. Los valores negativos chicos ($\geq -0,01$) son ajustes por redondeo; los menores son anomalías.
S3	<code>subtotal</code> , <code>credits</code> y <code>taxes</code> de facturación están en la moneda <code>currency</code> . El USD se obtiene como $(\text{subtotal} - \text{credits} + \text{taxes}) \times \text{exchange_rate_to_usd}$.
S4	<code>credits</code> nulo equivale a 0.
S5	Los CSV de maestros son snapshots completos (no incrementales).
S6	El NPS de <code>customers_orgs.nps_score</code> es un score agregado en [-100,100]; el de <code>nps_surveys</code> es un resultado de encuesta, no una respuesta individual 0-10.
S7	El entorno de ejecución es Google Colab (2 vCPU, ~12 GB RAM) con persistencia en Google Drive, más AstraDB <i>free tier</i> .

10.2 Riesgos y mitigaciones

ID	Riesgo	Prob.	Impacto	Mitigación
K1	Un watermark corto descarta casi todos los eventos (97,5 % con 1 día)	Alta	Alto	Watermark de 62 días + quarantine de tardíos + recomputo por fecha (D-05)
K2	Semántica de FX ambigua (USD con FX $\neq$ 1; ARS a 0,0015)	Alta	Alto	Regla Q08, supuesto S3 documentado, validar con el docente; FX parametrizable
K3	Colab es efímero: se pierden checkpoints y el lake	Alta	Medio	Lake y <code>_checkpoints</code> en Google Drive montado; script de <code>reset</code>
K4	Conectividad o credenciales de AstraDB fallan durante la demo	Media	Alto	Secure bundle + token por variables de entorno; plan B: Cassandra en Docker; capturas de respaldo (Sección 8.4 de la consigna)
K5	Desfase por zona horaria (verificado: Spark en hora local corrió fechas a 02/07 y generó 1.102 claves de más)	Alta	Medio	<code>spark.sql.session.timeZone=UTC</code> obligatorio en config (D-07)
K6	<i>Small files</i> por particionar de más	Media	Bajo	Particionar sólo por fecha + <code>coalesce</code> (Sección 7.2)
K7	PII (<code>email</code> , recursos con <code>pii:true</code>) expuesta en Gold	Media	Medio	Hash SHA-256 del email en Silver; no se publica en Gold ni en Cassandra
K8	Trabajo individual: carga horaria alta, curva de aprendizaje de Streaming/Cassandra y una sola persona como punto único de falla	Media	Alto	Spikes técnicos tempranos (streaming y AstraDB en las primeras 2 semanas de la E2); backlog obligatorio/deseable; consultas tempranas al docente

10.3 Decisiones abiertas (a cerrar con el feedback de la Entrega 1)

ID	Pregunta	Opciones	Recomendación actual
A1	¿Cómo interpretar <code>exchange_rate_to_usd</code> para USD $\neq$ 1?	(a) aplicarlo igual; (b) forzar 1,0 para USD; (c) usar una tabla FX externa	(a) con flag Q08
A2	Watermark y late data (D-05)	62 días / merge en Silver / sin watermark	62 días + quarantine de tardíos
A3	Formato de tabla	Parquet puro / Delta Lake	Parquet (lo pide la consigna); Delta como mejora opcional
A4	Componente analítico/ML	z robusto/MAD (sin entrenamiento) / IsolationForest MLlib	MAD por servicio+métrica como base
A5	Carga a Cassandra	Spark Cassandra Connector / <code>cassandra-driver</code> en <code>foreachBatch</code>	Driver en <code>foreachBatch</code> (más simple en Colab con AstraDB)

10.4 Modelo de serving preliminar (query-first, a validar en la Entrega 2)

Consulta obligatoria (consigna, Sección 7.4)	Tabla	PRIMARY KEY
Q1 Costos y requests diarios por org y servicio en un rango de fechas	usage_by_org_day_service	((org_id), usage_date, service)
Q2 Top-N servicios por costo acumulado en 14 días	cost_14d_by_org_service	((org_id), cost_14d_usd, service) – clustering DESC por costo
Q3 Tickets críticos y SLA breach por día (30 días)	tickets_by_severity_day	((severity), date) – sólo 4 particiones, clustering por fecha
Q4 Revenue mensual con créditos e impuestos en USD	revenue_by_org_month	((org_id), month)
Q5 Tokens GenAI y costo estimado por día	genai_by_org_day	((org_id), date)

11. Estimación preliminar de esfuerzo, roles y recursos (consigna, Sección 5.2.11)

11.1 Roles

El trabajo es **individual**: Katia Menshikoff (legajo 64396) asume todos los roles. Igual se separan las responsabilidades por rol, porque ordenan el trabajo y el backlog, y porque fijan quién es owner de cada dominio (gobierno, Sección 7.6).

Rol (lo cubre Katia Menshikoff)	Responsabilidad	Dominios
Arquitecta de datos	Arquitectura, decisiones, documentación, integración y defensa	Gobierno, metadatos
Data Engineer batch + calidad	Ingesta batch, Silver, reglas de calidad, marts de facturación y soporte	FinOps-revenue, Soporte
Data Engineer streaming + serving	Structured Streaming, marts de uso y GenAI, Cassandra/AstraDB, anomalías	FinOps-uso, Producto

11.2 Esfuerzo estimado (horas)

Fase	Entregable	Horas	Semanas disponibles	Horas/semana
E1 · Diseño y fundación (hasta 05/10)	Este documento, repo, perfil, MapReduce	30	–	–
E2 · Bronze batch + streaming	3 maestros + eventos, checkpoints	24		
E2 · Silver + calidad + quarantine	v1/v2, joins, Q01-Q11	20		
E2 · Gold + Cassandra + idempotencia	mart diario + 2 consultas	20		
E2 · Gobierno + README + evidencias + backlog		12		
Subtotal E2 (05/10 → 16/11)		76	6	~13
Final · Marts restantes, 5 consultas, ML/anomalías		26		
Final · Pruebas, documentación, video, defensa		22		
Subtotal final (16/11 → 07/12)		48	3	~16
Total		154		

Se suma un 20 % de contingencia (~30 h), concentrada en streaming y AstraDB. Con una sola persona la coordinación es mínima, pero no hay a quién repartir el trabajo. Por eso el backlog se ordena en **obligatorio** / **deseable** / **fuera de alcance** desde la Entrega 2, y lo deseable (por ejemplo Delta Lake o IsolationForest) sólo se encara si lo obligatorio está cerrado.

11.3 Recursos

Recurso	Uso	Costo
Google Colab (CPU estándar) + Google Drive	Ejecución de PySpark y persistencia del lake	Gratis
PySpark 4.x, Java 17/21	Motor	Libre
DataStax AstraDB (free tier) o Cassandra 5 en Docker	Serving	Gratis
GitHub (repo privado)	Versionado, issues como backlog	Gratis

12. Repositorio y evidencia (consigna, Sección 5.2.12)

- Estructura según la Sección 8.1 de la consigna: `README.md`, `docs/`, `data/`, `src/`, `notebooks/`, `tests/`, `config/`, `infra/`, `evidence/`, `DECISIONS.md`.
- Evidencia mínima de lectura y exploración:
 - `evidence/profile/profile_report.md` y `profile_summary.json`: perfil de las 8 fuentes.

- `evidence/profile/landing_manifest.sha256` : huella de Landing para verificar que no se modifica.
- `evidence/mapreduce/*` : salida y contadores del MapReduce, salida Spark y plan físico.
- `notebooks/00_exploracion_landing.ipynb` : lectura con PySpark y esquema explícito (Colab).
- `tests/` : 6 pruebas unitarias del flujo MapReduce (`pytest`).

■ Próximos pasos (hacia la Entrega 2)

1. Incorporar el feedback de la Entrega 1 en el **plan de correcciones** (`docs/03_plan_correcciones.md`).
2. Implementar Bronze batch para `customers_orgs`, `users`, `billing_monthly`, y Bronze streaming para eventos.
3. Silver de eventos y reglas Q01–Q05; mart Gold `org_daily_usage_by_service`.
4. Keyspace `cpa` en AstraDB, tabla `usage_by_org_day_service` y consultas Q1/Q2.